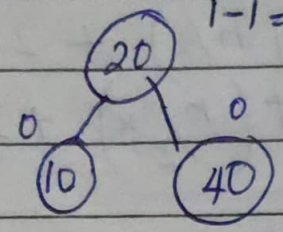
MODULE 2 :-

ADVANCED DATA STRUCTURES AND GRAPH ALGORITHM

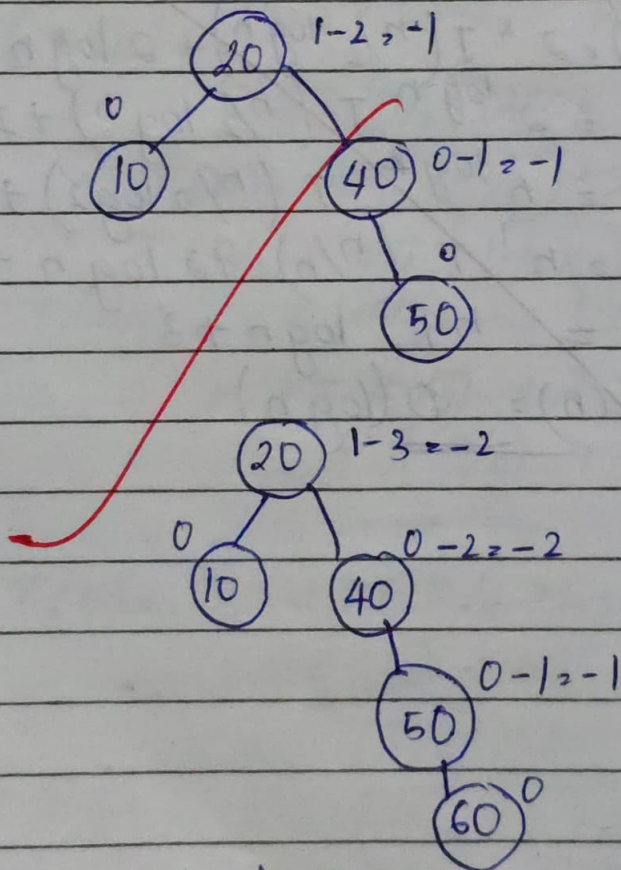
AVL TREE :-



Values of Balance factor ⇒ 0, 1, -1

⇓

Height of left Subtree - Height of right Subtree

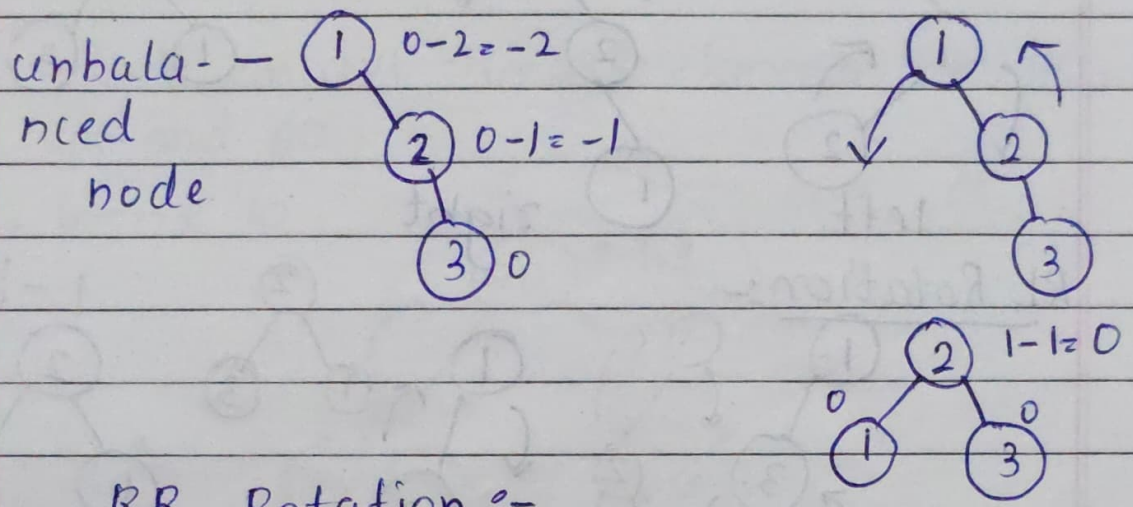


Rotation ⇒ single Rotation → RR, LL

Double Rotation ⇒ RL, LR

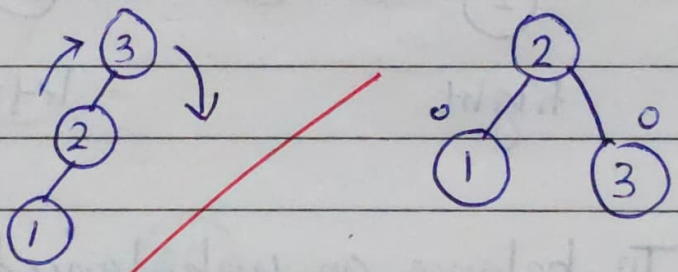
LL Rotation :-

Insertions → right



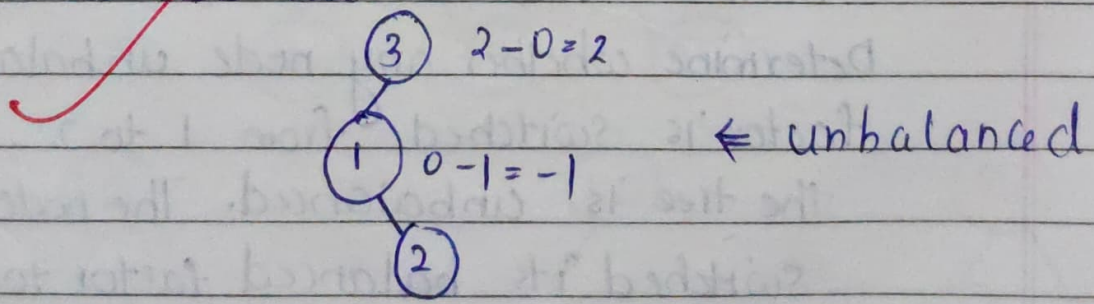
RR Rotation :-

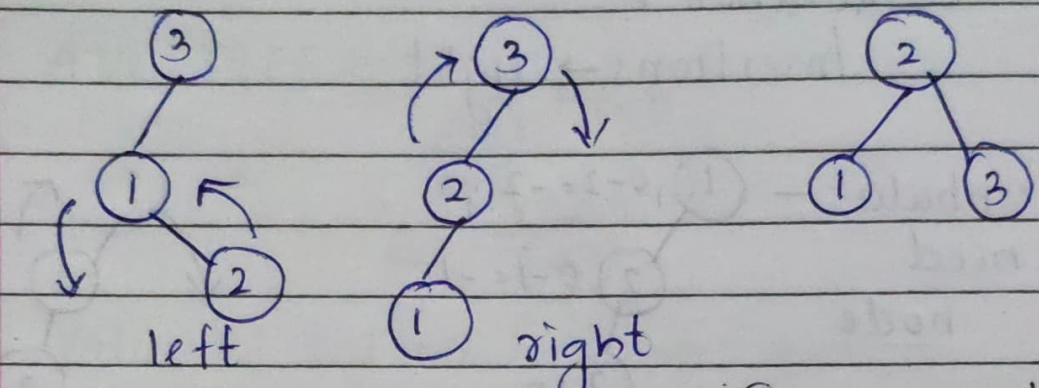
Insertions : left



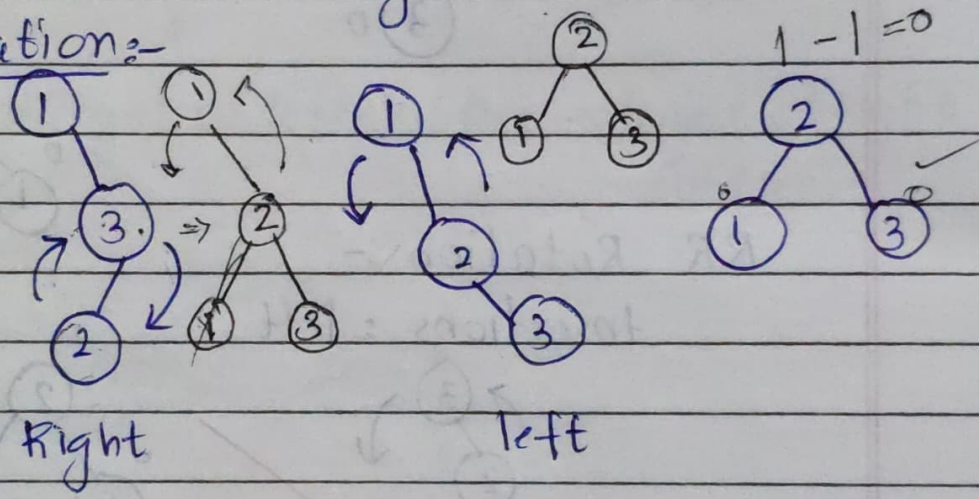
LR Rotation :-

Insert 3, 1, 2 into a BST and make it as AVL tree





RL Rotation:-



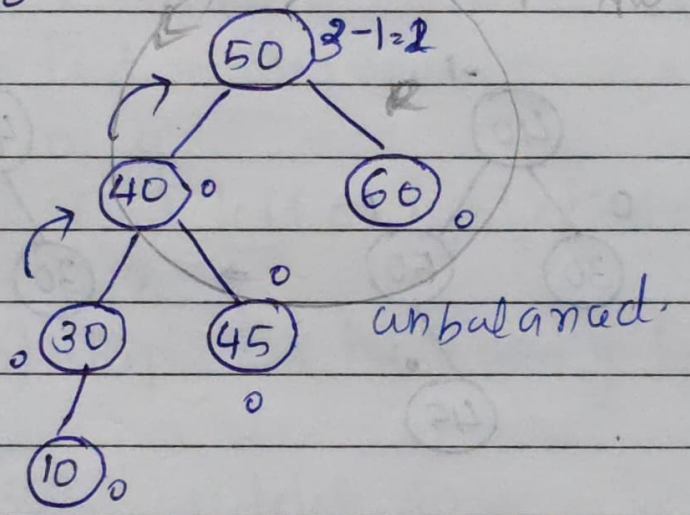
To balance an unbalanced Tree:-

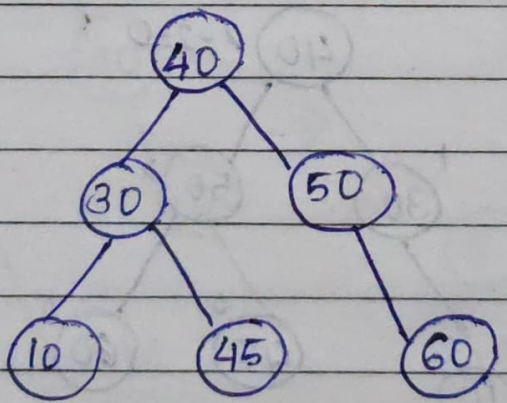
1. Insert node into BST.
2. Compute the balance factor.
3. Decide the pivot node, after step 2 Determine whether any node unbalanced factor is switched from 1 to 2, If so the tree is unbalanced. The node which switched its balanced factor to 2 is called pivot node.

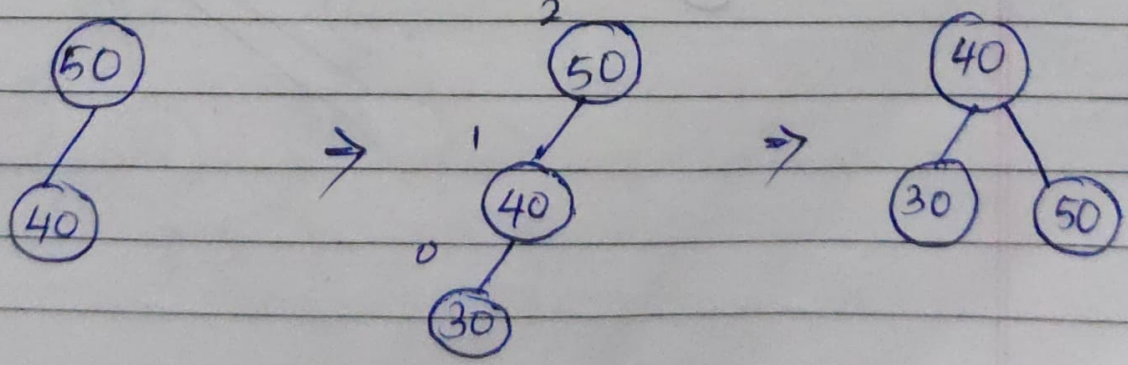
step 4: Balance the unbalanced tree using the procedure.

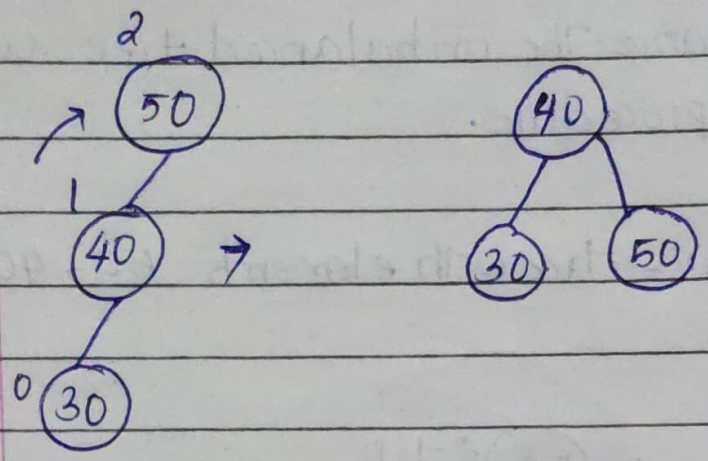
Q. Create an AVL tree with elements 50, 40, 30, 45 and 60.

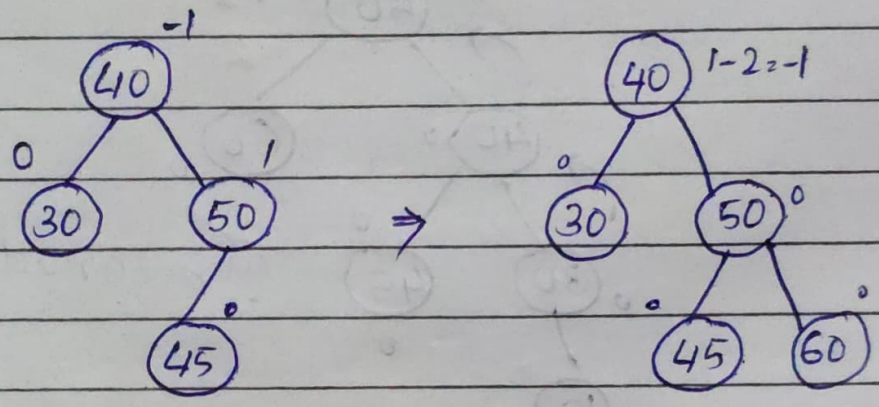
- Insert 10

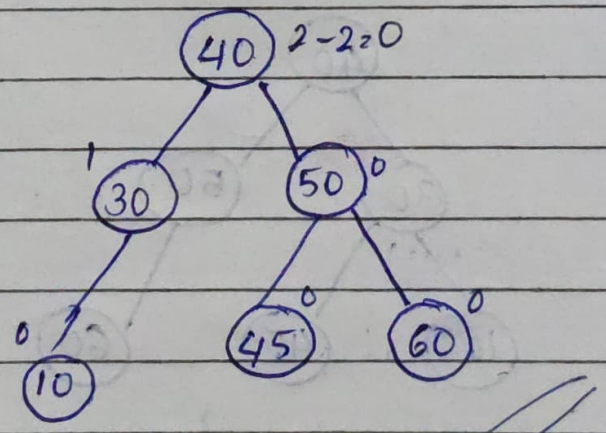







2
50
1
40
0
30
40
30
50


-1
40
0
1
30
50
0
45
40
1-2=-1
0
30
50
0
0
0
45
60


40
2-2=0
1
30
50
0
0
0
10
45
60

Algorithm of BST deletion :-

- Let x be the node to be deleted.

Case I : x is a leaf node. delete x

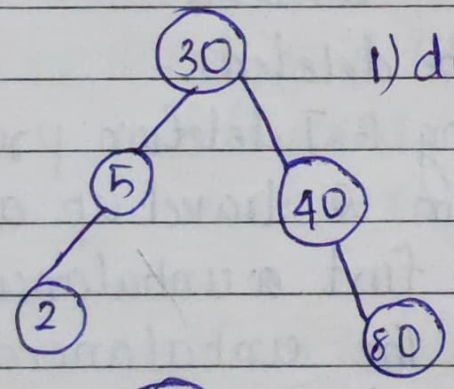
case II : x is a node with only 1 child, Replace x with child.

case III : x having 2 children,

i) find 'inorder successor of a x it called as y.

ii) Find left or right child of y saying z.

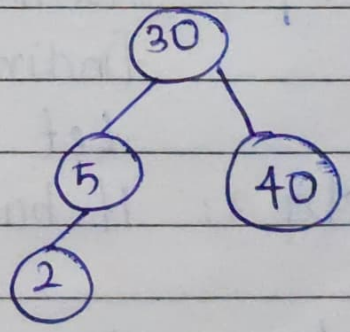
iii) Replace x by y and y by z.



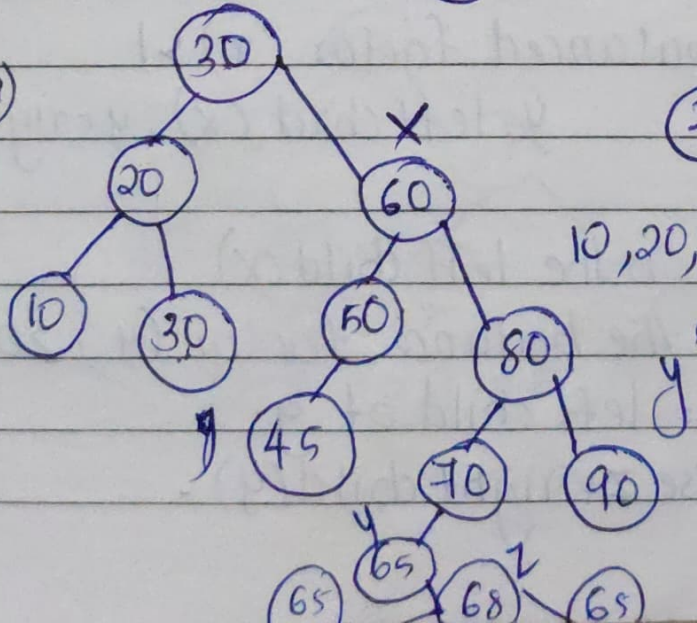
1) delete 40

Replace 40 with 80.

⇒



1)



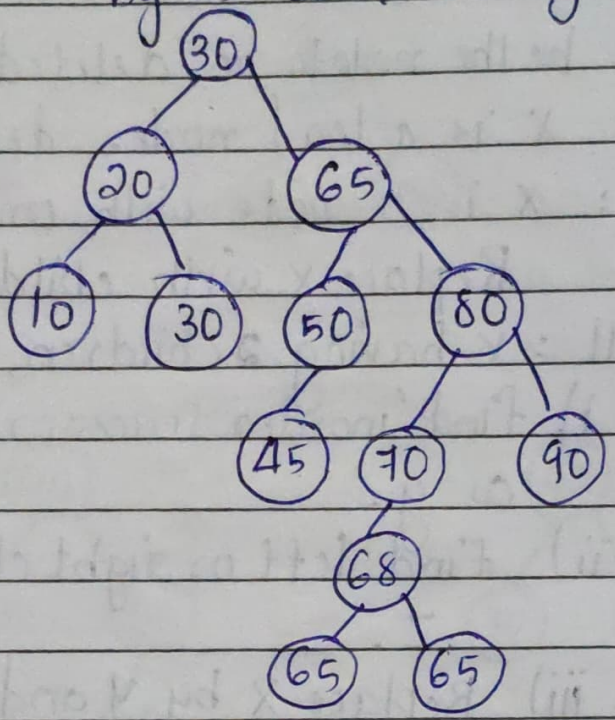
10, 20, 30, 30, 45, 50, 60, 65, 70

X

y

Replace 60 by 65 and 65 by 68

→



Algorithm for AVL Tree Deletion:-

Let W be the node to be deleted.

Step 1: Delete W using BST deletion procedure.

Step 2: Starting from W travel up and finding the first unbalanced node. Let X be the unbalanced node.

step 3: If balanced factor (x) > 1
y = left child (x), y = right child (x)

step 4: else

step 5: If y is the left child (x)

5.1: If the balance factor (y) ≥ 0, then z = left child of y.

5.2: else z = right child (y).

step 6: else

6.1: If balance factor < 1 z = right child(y)

6.2: z = right child(y)

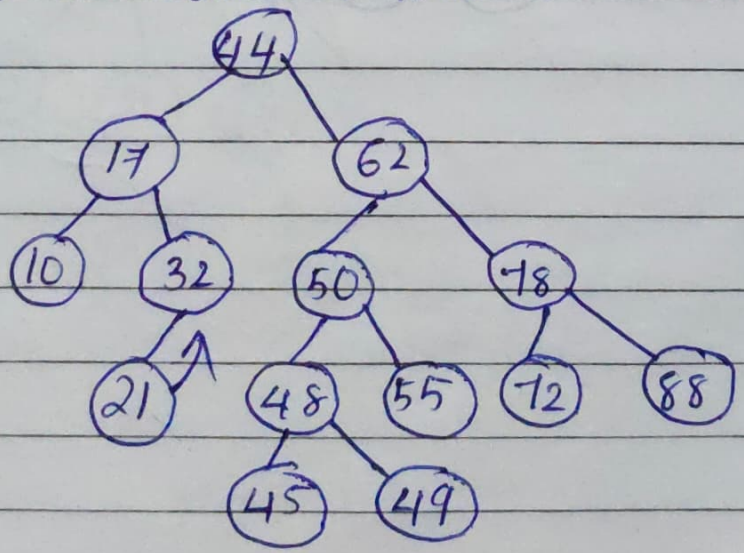
step 7: y is the left child of x and z is the left child of y then perform RR rotation wrt to x.

step 8: If y is the right child(x) and z is the right child(y), then perform LL rotation wrt to x.

step 9: If y is the left child(x) and z is the right child(y) then perform LR rotation wrt to x.

step 10: If y is the right child(x) & z is the left child(y) then perform RL rotatn wrt to x.

Q. Delete 32 from the tree?



1) X 44 2-0=-2 44 = unbalanced = X

1-1=0

17 y 62 3-2=1

10 21 2-1=1 78

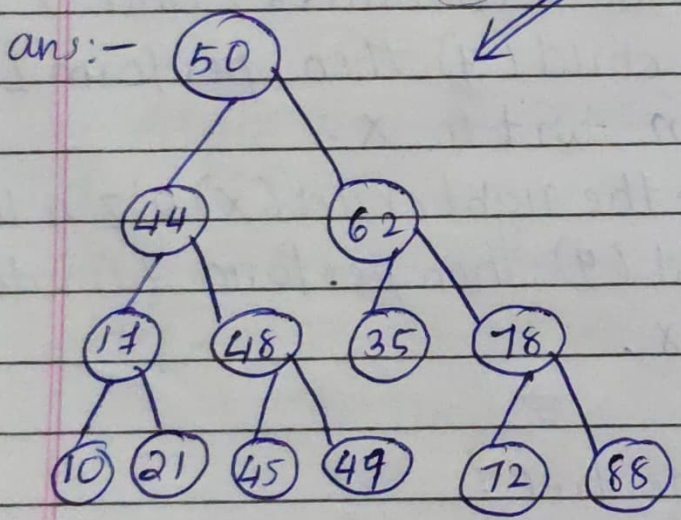
RL Rotation

highest height z 50

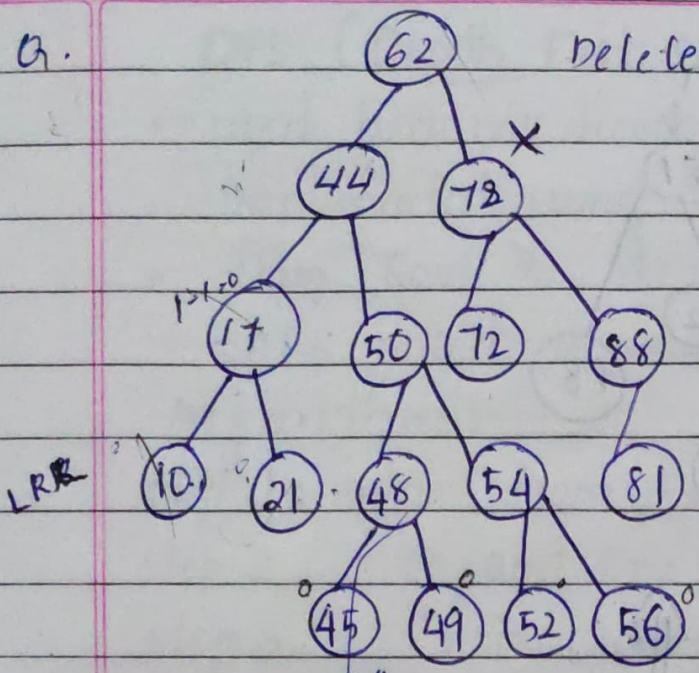
40 55 72 88

45 49

ans:-

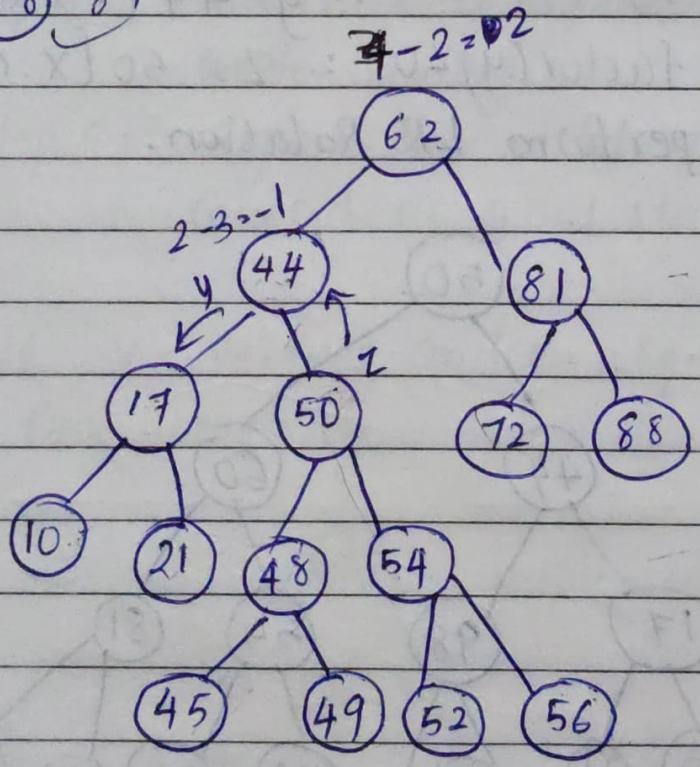


Q. Delete 78 from tree?



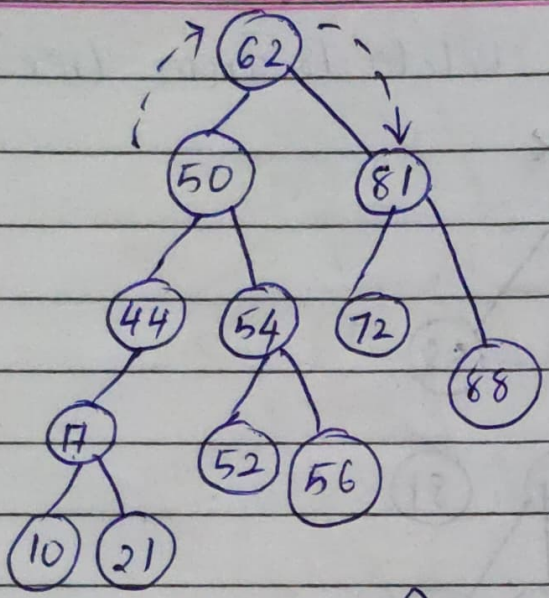
Inorder Successor :-

10 17 21 44 45 48 49 50 52 54 56 62 72 78 81



X = 78

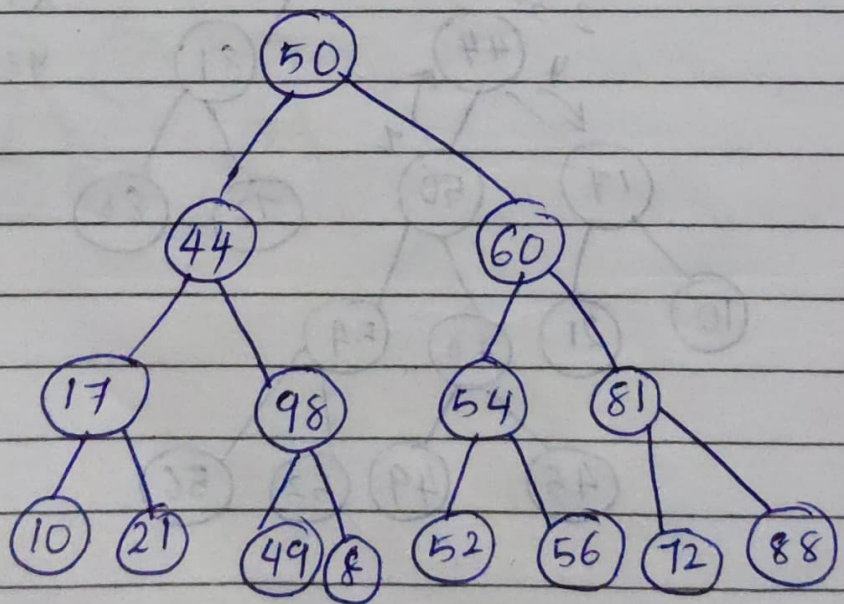
y = 81



b.f(x) > 1 ∴ y = 44 [l.c(x)]

balance factor(y) < 0 ∴ z = 50 [x.c(y)]

So perform LR Rotation.



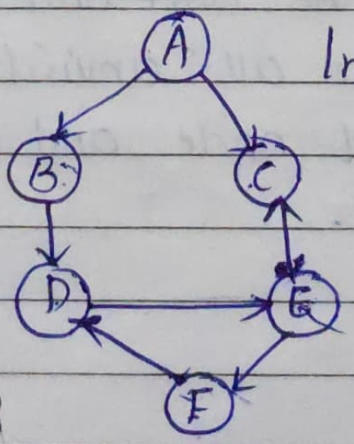
DFS (Depth First search)

- used heuristic function.
- Implemented using stack.
- From Root to depth.
- Also called Incomplete search method.
- time Complexity $O(b^d)$

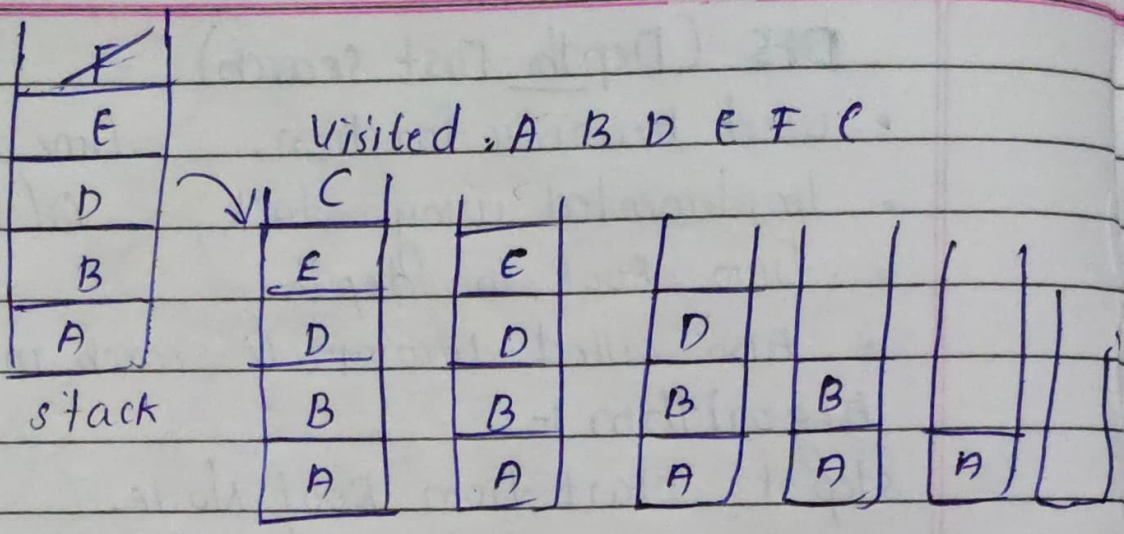
Algorithm :-

step 1: start from Root Node.
step 2: Expand one of the child nodes of root node.
step 3: Explore child node. (go deeper until)
  3.1: A solution is found. (goal state)
  3.2: Branch ends
step 4: Backtrack to explore unexplored ~~node~~. Siblings.
step 5: Repeat this until all node is covered or goal state is found.

- If V - vertices and E - edges. then time Complexity = $O(V+E)$

Q. 

Implement DFS.

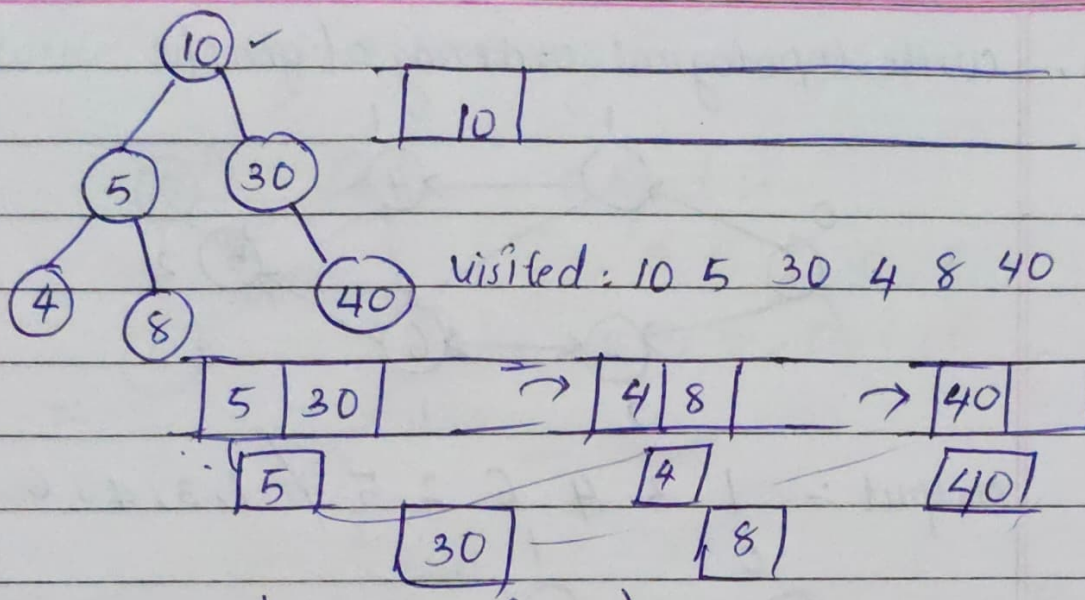


BFS (Breadth First Search)

- Traversal algo
- level by level

Algorithm:

1. Initialise a queue and enqueue the starting node.
2. Mark the starting node as visited.
3. while the Queue is not empty.
   - 3.1 : Dequeue the node from the Queue.
   - 3.2 : process the dequed neighbours.
   - 3.3 : ~~process~~ enqueue ~~the~~ all ~~node and mark them~~
   - 3.3 : Enqueue all unvisited neighbors of dequed node and mark them as visited.



Time Complexity = O (V+E)

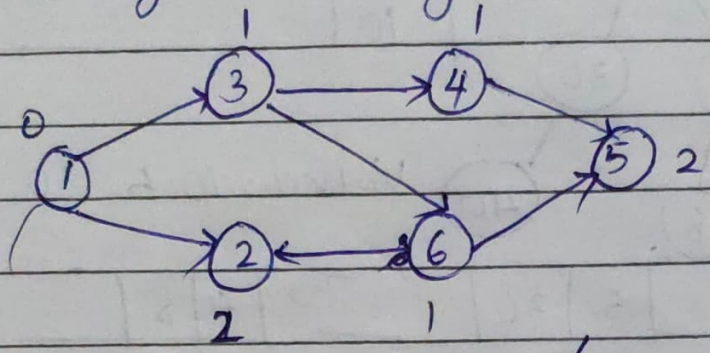
Topological Sorting / ordering

- DAG (Directed Acyclic graph)

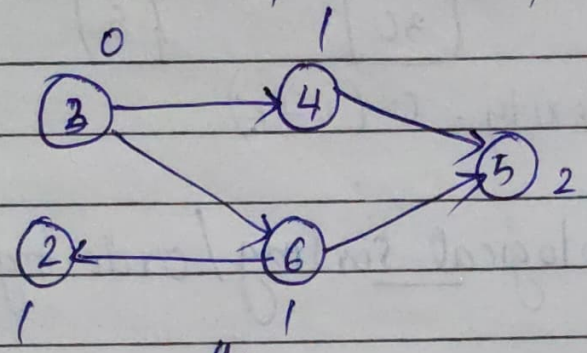
ALGORITHM :-

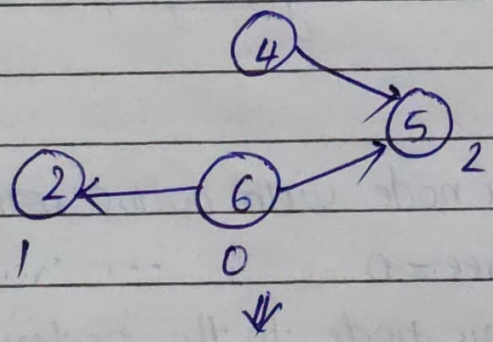
1. Identify a node with no incoming edge or indegree = 0.
2. Add this node to the ordering.
3. Remove this node and all it's outgoing edges from the graph.
4. Repeat 1 to 3 until graph become empty.

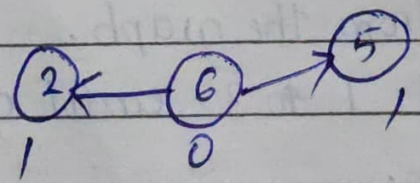
Q. write topological ordering of graph.



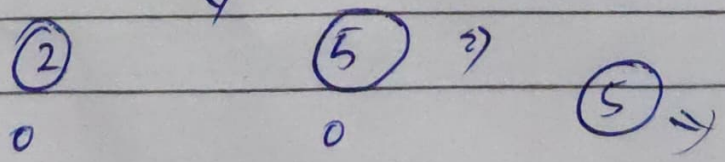
Output :- 1 3 4 6 2 5 / 1, 3, 4, 6, 5, 2



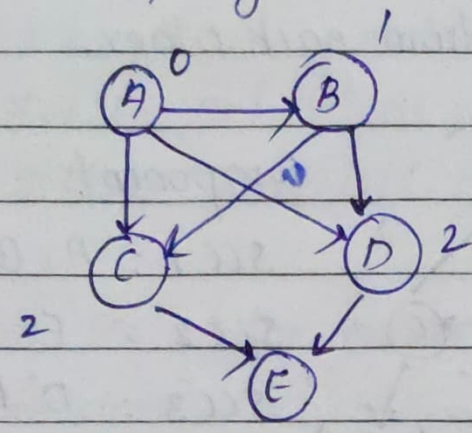




Delete 2



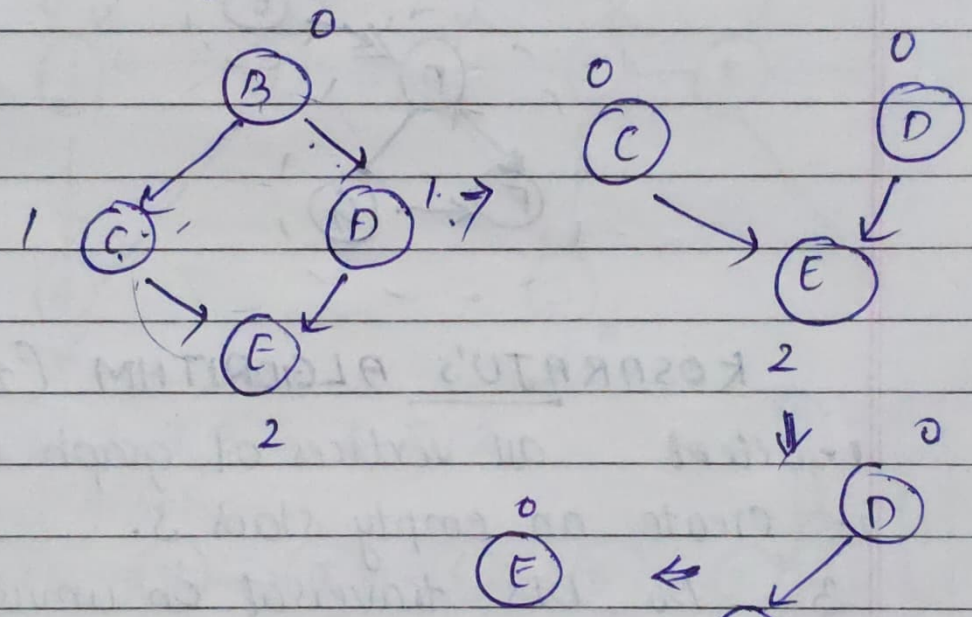
Q. write topological ordering for the graph.



Output :-

A, B, C, D, E

A, B, D, C, E



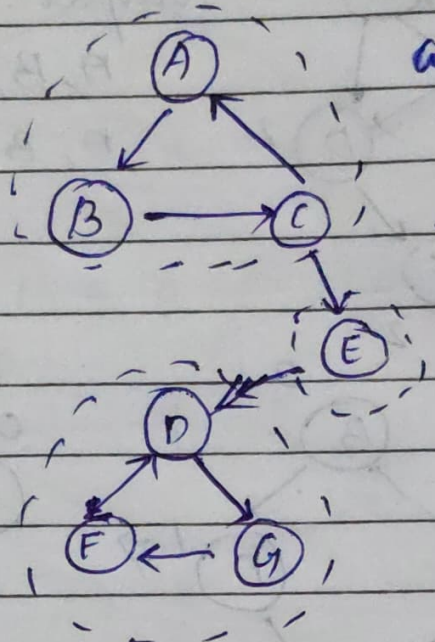
STRONGLY CONNECTED COMPONENTS (SCC)

- It applies only to directed graph
- A directed graph is strongly connected if there is a path from each vertex to another vertex.

ie, for every pair of vertex (v, u) there will be

a path from U to V and V to U. ie Both u and v are reachable from each other.

eg:-



Components

SCC 1 : A, B, C

SCC 2 : E

SCC 3 : D F G

KOSARAJU'S ALGORITHM (2 pass Algo)

1- Set all vertices of graph G are unvisited.

2- Create an empty stack S.

3. Do DFS traversal on unvisited vertices and set it as visited. If a vertex has no unvisited neighbors push it into the stack.

4. perform the above steps until all vertices are visited.

5. Reverse the graph G^T.

6. Set all nodes are unvisited.

7. while S is not empty

7.1.1: pop one vertex v'.

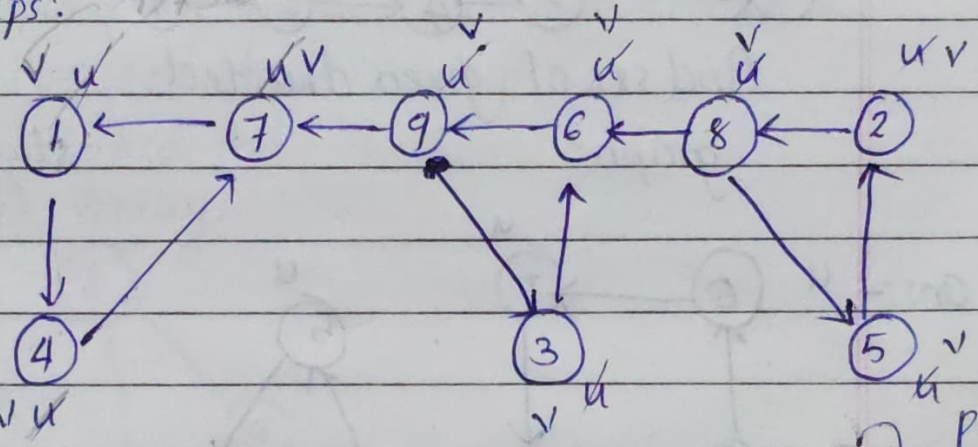
7.1.2: If v' is not visited

7.2.1: Set v' as visited.

7.2.2: Call DFS of v'. It will print scc of v'.

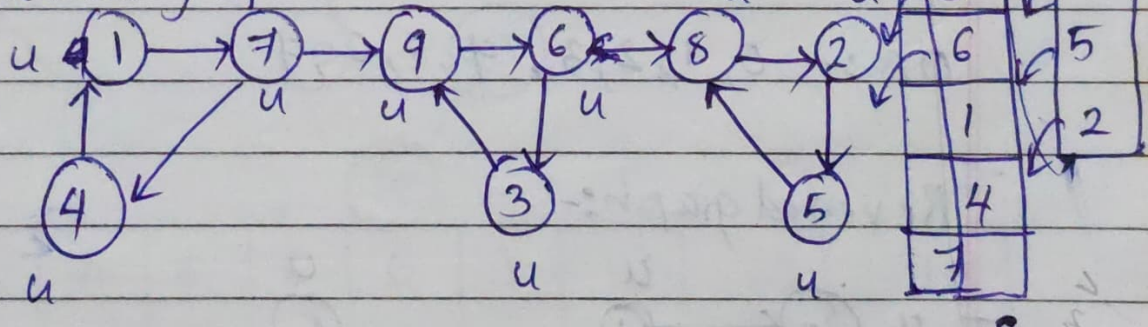
Imp

Q. Find scc of the graph using algorithm by showing each steps?



DFS: 1, 4, 7, 9, 3, 6, 8, 5, 2

Reversed graph:-



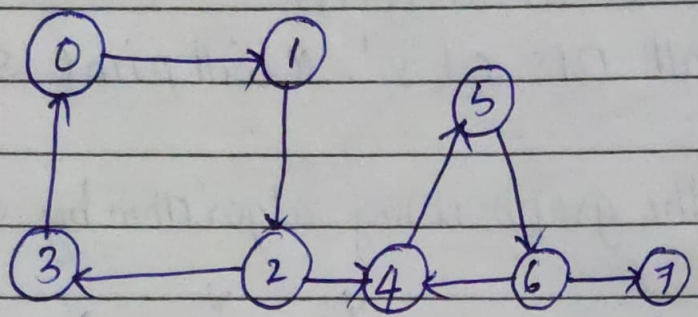
DFS: (8, 2, 5), (9, 6, 3), (1, 7, 4)

stronglyconnectedcomponent.

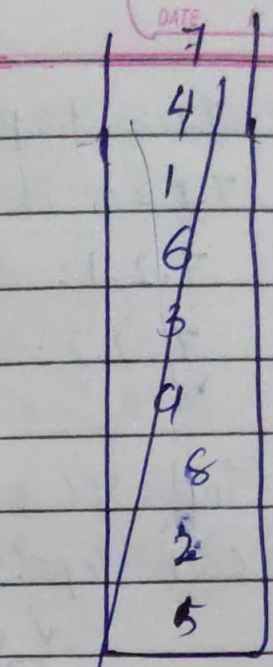
SCC1 = 8, 2, 5

SCC2 = 9, 6, 3

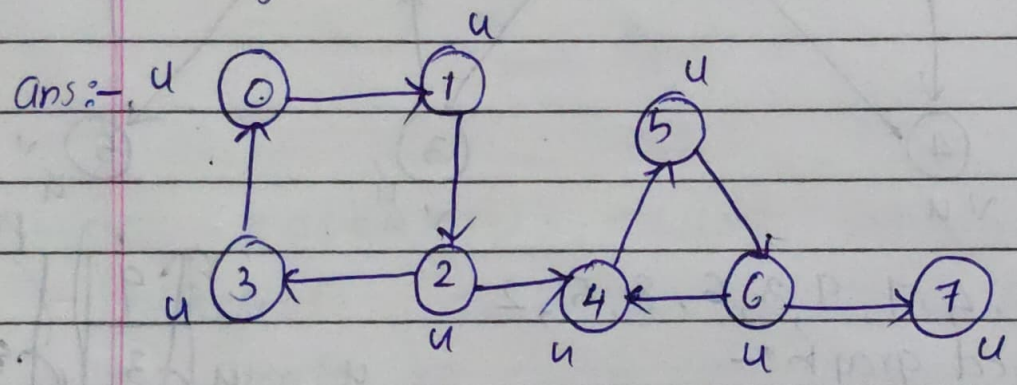
SCC3 = 1, 7, 4

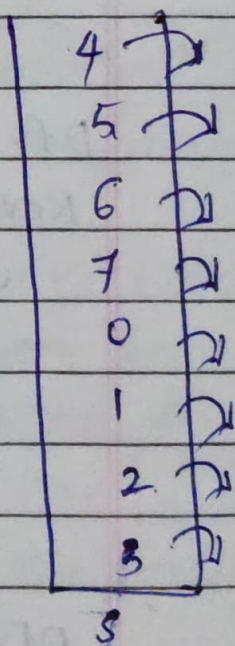
Q. 

find scc of given directed graph?



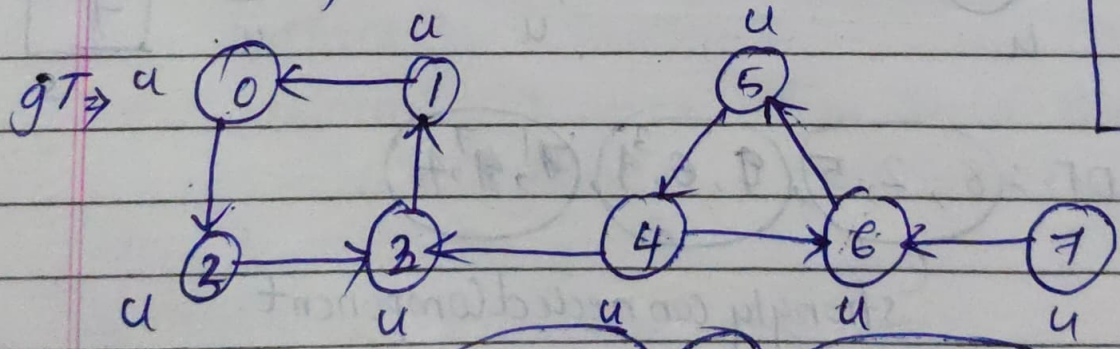
stack :-

ans :- 



DFS :- 0, 1, 2, 3, 4, 5, 6, 7

Reversed graph :-

gT ⇒ 

~~DFS~~ SEC = (4, 6, 5) (7) (0, 3, 2, 1)

SCC 1 = 4, 6, 5

SCC 2 = 7

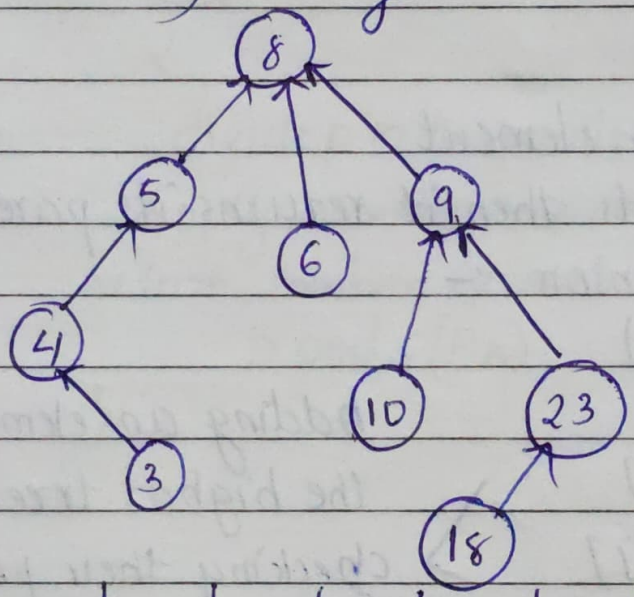
SCC 3 = 0, 3, 2, 1

DISJOINT SETS

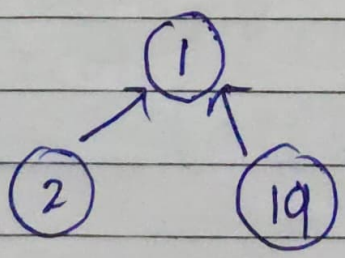
$S_1 = \{1, 2, 3\}$ $S_2 = \{4, 5\}$

Does not contain common elements.

- It is represented in 2 ways:
  1) linked list
  2) Array



3	4	5	8	6	9	10	23	18
4	5	8	-1	8	8	9	9	23



2	1	19
1	-1	1

OPERATIONS OF DISJOINT SETS :-

1. Make set (x) - New sets are created using x.
2. union (x, y)
3. find (x) - Searching for x.

- Algorithm of find :-

```
find(i)
{
  while p[i] > 0 do        checking parent node.
      i = p[i]                  " -1 > 0
  return i                      " then PN.
}
```

- Searching an element
  If it exists then it returns its parent node.

- Algorithm of union :-

```
union (i, j)
{
  x = find[i]
  y = find[j]
If x != y then
    p(y) = x
}
```

Adding an element to the highest tree.
checking their parents are equal or not

Applications :-

- Identify scc easily.